

Python Data Structures & Why They're Slow for Numbers

List, Tuple, Dict, Set | Motivation for NumPy

In Python, we have several built-in data structures — **list**, **tuple**, **dict**, and **set** — each used to store a different kind of data.

1. Built-in Data Structures

1.1 List

A list can store **homogeneous** (same type) as well as **heterogeneous** (mixed type) data.

```
script.py

l1 = [10, 20, 30, 40] # list of numbers (homogeneous)

l2 = ["shahid", "raj", "priyanshu", "kashish"] # list of strings
# storing related data (homogeneous)

l3 = [{"name": "md shahid"}, {"age": 24}, {"college": "iit"}]
# list of objects (dictionaries)
```

1.2 Dictionary

A dictionary is used to store related data as key-value pairs.

```
script.py

student = {
    "name": "raj",
    "age": 24,
    "email": "raj@example.com"
}
```

1.3 Set

A set is used to store **unique** data — duplicate values are automatically removed.

```
script.py

unique_ids = {101, 102, 103}
```

Note: list3 (l3) is a list containing dictionaries as its elements — each dictionary happens to hold just one key-value pair here, so it behaves like a list of small records.

2. The Problem — Numerical Performance

Python's built-in data structures are convenient and flexible, but they have real drawbacks when used for heavy numerical work:

- Slower for numerical calculations
- Take up more memory
- So they are not efficient for numerical calculations

3. Demonstrating the Slowness

Let's see why Python's data structures are not efficient for numerical calculations when we have a massive amount of data.

3.1 Create a list of n natural numbers

```
script.py

import time

list = []
for i in range(1, 1001):
    list.append(i)

# print(list)
```

3.2 Calculate the sum and time it

```
script.py

sum = 0
start = time.time()

for i in range(0, 1000):
    sum = sum + list[i]

end = time.time()
print(sum)
print(f"time taken:{end-start:.6f}")
```

Note: Avoid naming variables `list` and `sum` — they shadow Python's built-in `list()` and `sum()` functions. Prefer names like `numbers` and `total` instead.

4. Conclusion

The more numerical data we have, the more calculation time it takes when using plain Python loops over a list.

This means: if we have a large amount of numerical data — as is common in **data analytics** and **data science** — and we need to perform mathematical calculations on it, we should **not** rely on plain Python data structures like list. Instead, we should use a **NumPy array**.

Quick Summary

Structure	Stores	Ordered?	Duplicates?	Mutable?
list	Homogeneous or heterogeneous data	Yes	Yes	Yes
tuple	Homogeneous or heterogeneous data	Yes	Yes	No
dict	Key-value pairs	Yes (insertion order)	Keys: No	Yes
set	Unique values only	No	No	Yes

Note: Why NumPy is faster: A NumPy array stores elements of a single fixed type in one contiguous block of memory and performs operations using vectorized, compiled C code — instead of looping element-by-element in the Python interpreter like a plain list does. This gives large speed and memory advantages for numerical work.